

Proyecto IU-Ayuntamiento

Ángela Moya García
Kevin González Sánchez
Aitor Inarejos Hedrera
Jose Román Castellano

25 de mayo de 2026

Repositorio del proyecto: <https://github.com/Kevings7/TW-Practica>

Índice

1. Introducción	3
2. Idea general del proyecto	3
3. Parte estática: IU-Ayuntamiento	3
3.1. Estructura de carpetas de la parte estática	3
3.2. Archivos y carpetas de la parte estática	4
3.3. Cómo se hizo la parte estática	5
3.4. Cómo se ejecuta la parte estática	5
4. Base de datos	6
4.1. Funcionamiento con la base de datos	7
5. Parte dinámica: paso a Laravel	7
5.1. IU-Ayuntamiento-Laravel	8
5.1.1. Estructura de carpetas en Laravel	8
5.1.2. Explicación de las nuevas carpetas y archivos	9

5.2. Rutas, métodos GET/POST y middleware	11
5.3. Modelo, vista y controlador en Laravel	13
5.4. Cómo se han hecho las migraciones	15
5.5. Funcionamiento de la parte dinámica	19
5.6. Diseño, accesibilidad y responsive	20
5.7. Funciones disponibles en la parte dinámica	21
6. Conclusiones	22

1. Introducción

En este documento vamos a explicar cómo se hizo el proyecto dividiéndolo en tres partes principales: la parte estática, la parte dinámica y la base de datos. Primero se explica la versión IU-Ayuntamiento, hecha con PHP, HTML, Bootstrap, CSS y MySQL. Después se explica la base de datos, que es común a la lógica del proyecto. Por último, se explica cómo pasamos la aplicación a una versión dinámica con Laravel en IU-Ayuntamiento-Laravel.

2. Idea general del proyecto

Este proyecto consiste en una página de gestión de incidencias urbanas para el ayuntamiento de Granada. La web permite ver incidencias públicamente, registrarse, iniciar sesión, crear incidencias y, cuando el usuario tiene rol técnico, entrar a un panel para gestionar el estado de las incidencias.

Lo hicimos en dos versiones. Primero está la versión normal con PHP, HTML, Bootstrap, CSS y MySQL. Después está la versión con Laravel, donde la misma idea se organiza mejor usando rutas, controladores, modelos, migraciones y vistas Blade.

3. Parte estática: IU-Ayuntamiento

La parte estática corresponde a la primera versión del proyecto, hecha con PHP a pelo, HTML, Bootstrap y CSS. Aquí no hay Laravel. Cada página es un archivo PHP independiente, y para no repetir todo el HTML común se usan archivos dentro de `includes`. Aunque esta parte ya consulta datos, la organización es más simple y está basada en archivos sueltos.

3.1. Estructura de carpetas de la parte estática

```
IU-Ayuntamiento/  
|-- contacto.php  
|-- css/  
|   '-- estilos.css  
|-- detalle.php  
|-- gestionar_incidencia.php  
|-- incidencias.php  
|-- includes/  
|   |-- conexion.php  
|   |-- footer.php  
|   '-- header.php  
|-- index.php  
|-- login.php  
|-- logout.php  
|-- mis_incidencias.php  
|-- panel_tecnico.php
```

```
|-- register.php
|-- reportar.php
|-- sql/
|   |-- creacion_tablas.sql
|   '-- datos.sql
'-- uploads/
    '-- incidencias/
```

3.2. Archivos y carpetas de la parte estática

Archivos y carpetas	Explicación
contacto.php	Muestra el formulario de contacto, pero solo si el usuario está identificado. Si no hay usuario, muestra un aviso de acceso restringido.
css/estilos.css	Contiene el estilo propio de la web. Se usa para cambiar colores, tarjetas, barra de navegación, banner, footer y formularios. Está cargado desde el header.
detalle.php	Muestra el detalle completo de una incidencia concreta. Recibe el id por GET y consulta la base de datos con JOIN para traer tipo, estado, prioridad, barrio, usuario e imagen.
gestionar_incidencia.php	Página del técnico para cambiar el estado de una incidencia. Comprueba sesión, comprueba que el rol sea técnico, actualiza la incidencia, guarda historial y crea una notificación.
incidencias.php	Lista las incidencias en formato de tarjetas. Es una vista pública de todas las incidencias visibles.
includes/conexion.php	Archivo común para conectar con la base de datos MySQL <code>gestion_incidencias</code> .
includes/header.php	Cabecera común. Inicia sesión si hace falta, carga Bootstrap, carga el CSS propio y monta el menú según el usuario y su rol.
includes/footer.php	Pie de página común. Se incluye al final de las páginas para no repetir el mismo HTML.
index.php	Página principal. Carga categorías y las últimas incidencias para mostrarlas como página de inicio.
login.php	Formulario e implementación de inicio de sesión. Busca el usuario por email, comprueba la contraseña y guarda datos en la sesión.
logout.php	Cierra la sesión y redirige al inicio.
mis_incidencias.php	Muestra al ciudadano las incidencias que ha creado él mismo.
panel_tecnico.php	Muestra al técnico una tabla con las incidencias y accesos para gestionarlas.

Archivos y carpetas	Explicación
<code>register.php</code>	Formulario e implementación de registro. Inserta nuevos usuarios con rol ciudadano.
<code>reportar.php</code>	Formulario para crear una incidencia. Valida que el usuario esté logueado, carga tipos y barrios, guarda los datos y sube la imagen.
<code>sql/creacion_tablas.sql</code>	Script que crea la base de datos y todas las tablas necesarias.
<code>sql/datos.sql</code>	Script que inserta datos iniciales: roles, departamentos, tipos de incidencia, estados, prioridades y barrios.
<code>uploads/incidencias/</code>	Carpeta donde se guardan las imágenes subidas en las incidencias.

3.3. Cómo se hizo la parte estática

Primero hicimos la estructura básica de la web con páginas PHP separadas. Para no repetir la cabecera y pie de página en todos los archivos pusimos la cabecera en `includes/header.php` y el footer en `includes/footer.php`. Así, cada página solo tiene que hacer `include` y ya mantiene el mismo menú y diseño.

Una vez hecho esto añadimos la base de datos. En `sql/creacion_tablas.sql` se crean las tablas principales. La tabla más importante es `incidencias`, porque relaciona al usuario que crea el aviso, el tipo de incidencia, el estado, la prioridad y el barrio. Luego en `sql/datos.sql` se insertan datos iniciales para poder probar la web sin tener que meter todo a mano.

Después conectamos PHP con MySQL usando `includes/conexion.php`. Con esto ya las páginas pueden hacer consultas a la base de datos.

Luego se hizo el flujo de usuario: registro, login y logout. En `register.php` se crea el usuario nuevo como ciudadano, en `login.php` se comprueba el email y la contraseña, y en `logout.php` se destruye la sesión. La sesión sirve para saber si el usuario está identificado y qué rol tiene.

Cuando ya tenemos para poder iniciar sesión se añadió `reportar.php` para que un ciudadano pueda registrar una incidencia. Ahí se cargan los tipos y barrios desde la base de datos, se validan los campos y se guarda también la foto en la carpeta `uploads/incidencias`.

Finalmente se añade la parte del técnico. El técnico tiene un panel propio, `panel_tecnico.php`, y desde `gestionar_incidencia.php` puede cambiar el estado de una incidencia. Además, cuando se cambia el estado, se guarda el cambio en `historial_estados` y se crea una notificación para el ciudadano.

3.4. Cómo se ejecuta la parte estática

1. Abrimos XAMPP y activamos Apache y MySQL.
2. Entramos en phpMyAdmin.

3. Ejecutamos `sql/creacion_tablas.sql`.
4. Ejecutamos `sql/datos.sql`.
5. Abrimos en el navegador: `http://localhost/TW-Practicas/TW-Practicas/IU-Ayuntamiento`.

4. Base de datos

La base de datos es una parte separada del proyecto porque guarda toda la información importante de la aplicación. Se ha diseñado separando la información en varias tablas. La tabla principal es `incidencias`, porque representa cada aviso creado por un ciudadano. Esta tabla no guarda todos los datos directamente, sino que se relaciona con otras tablas mediante claves externas.

Por ejemplo, una incidencia tiene un usuario que la crea, un tipo de incidencia, un estado, una prioridad y un barrio. En vez de escribir el nombre del usuario, del estado o del barrio dentro de la incidencia, se guarda su identificador.

La relación principal sería:

- `roles` → `usuarios` → `incidencias`
- `departamentos` → `tipos_incidencia` → `incidencias`
- `estados_incidencia` → `incidencias`
- `prioridades` → `incidencias`
- `barrios` → `incidencias`
- `incidencias` → `imagenes_incidencia`
- `incidencias` → `historial_estados`
- `incidencias` → `comentarios_incidencia`
- `incidencias` → `notificaciones`

La tabla `incidencias` es la principal del proyecto. Las demás tablas guardan información relacionada con ella.

Tabla	Para qué sirve	Relación importante
<code>roles</code>	Guarda los tipos de usuario: ciudadano y técnico.	Se relaciona con <code>usuarios</code> .
<code>usuarios</code>	Guarda los datos de las personas registradas.	Cada usuario tiene un <code>rol_id</code> .
<code>departamentos</code>	Guarda las áreas del ayuntamiento.	Cada departamento tiene varios tipos de incidencia.

Tabla	Para qué sirve	Relación importante
<code>tipos_incidencia</code>	Guarda categorías como farola rota, bache, basura acumulada, etc.	Cada tipo pertenece a un departamento.
<code>estados_incidencia</code>	Guarda los estados: pendiente, validada, en proceso, solucionado y rechazada.	Cada incidencia tiene un estado.
<code>prioridades</code>	Guarda la urgencia de la incidencia.	Cada incidencia tiene una prioridad.
<code>barrios</code>	Guarda las zonas donde ocurre la incidencia.	Cada incidencia puede estar asociada a un barrio.
<code>incidencias</code>	Es la tabla principal. Guarda cada aviso urbano.	Se relaciona con usuario, tipo, estado, prioridad y barrio.
<code>imagenes_incidencia</code>	Guarda la ruta de las imágenes subidas.	Cada imagen pertenece a una incidencia.
<code>historial_estados</code>	Guarda los cambios de estado.	Permite saber cómo ha evolucionado una incidencia.
<code>comentarios_incidencia</code>	Guarda comentarios asociados a una incidencia.	Relaciona usuarios con incidencias.
<code>notificaciones</code>	Guarda avisos para el usuario.	Se usa cuando cambia el estado de una incidencia.

4.1. Funcionamiento con la base de datos

Un visitante puede entrar en la vista pública y consultar las incidencias que ya existen. Si quiere crear una incidencia, tiene que registrarse o iniciar sesión. Cuando inicia sesión, se guarda en `$_SESSION` su identificador, su nombre y su rol.

Si el usuario es ciudadano, el menú le muestra opciones como “Reportar incidencia” y “Mis incidencias”. Desde `reportar.php` puede rellenar un formulario con el título, descripción, tipo de incidencia, barrio, dirección, código postal, fecha e imagen. Al enviar el formulario, PHP valida los datos, guarda la incidencia en la base de datos y sube la imagen a la carpeta `uploads/incidencias`.

Si el usuario tiene rol técnico, al iniciar sesión se le redirige al panel técnico. Desde ahí puede ver todas las incidencias y entrar en `gestionar_incidencia.php` para cambiar su estado. Cuando cambia el estado, también se guarda un registro en `historial_estados` y se crea una notificación para el ciudadano.

5. Parte dinámica: paso a Laravel

La parte dinámica corresponde a la versión hecha con Laravel. Después de hacer la primera versión en PHP normal, pasamos el proyecto a Laravel para que la aplicación estuviera mejor organizada y fuera más fácil de mantener. En la parte inicial, cada pantalla dependía de un archivo PHP

independiente, como `index.php`, `login.php`, `reportar.php` o `panel_tecnico.php`. Eso funcionaba, pero mezclaba en muchos casos la lógica, las consultas a la base de datos y el HTML dentro del mismo archivo.

Con Laravel cambiamos esa forma de trabajar. La parte estática de la página, que antes era principalmente HTML con Bootstrap y algo de PHP, se transforma en una parte dinámica usando rutas, controladores, modelos y vistas Blade. Así, cuando el usuario entra en una URL, Laravel busca esa ruta en la carpeta `routes`, llama al controlador correspondiente, el controlador consulta los datos usando modelos y finalmente devuelve una vista desde `resources/views`.

La idea principal del cambio es separar responsabilidades:

- Las rutas indican qué URL existe y qué controlador debe responder.
- Los controladores contienen la lógica de la aplicación.
- Los modelos representan las tablas de la base de datos.
- Las migraciones crean y modifican las tablas de forma ordenada.
- Las vistas Blade muestran el HTML final al usuario.

Por ejemplo, en la versión PHP normal, para mostrar incidencias se entraba directamente en `incidencias.php`. En Laravel, en cambio, se define una ruta, esa ruta llama a un controlador de incidencias, el controlador obtiene las incidencias de la base de datos y después carga una vista Blade para enseñarlas en pantalla.

5.1. IU-Ayuntamiento-Laravel

La versión `IU-Ayuntamiento-Laravel` mantiene la misma idea del proyecto, pero usando la estructura propia de Laravel. Esto permite tener una aplicación más limpia, porque cada carpeta tiene una función concreta.

5.1.1. Estructura de carpetas en Laravel

Según la estructura del proyecto Laravel, las carpetas y archivos principales son los siguientes:

```
IU-Ayuntamiento-Laravel/  
|-- app/  
|-- bootstrap/  
|-- config/  
|-- database/  
|   |-- factories/  
|   |-- migrations/  
|   |-- seeders/  
|   '-- database.sqlite
```



```

|-- public/
|-- resources/
|   |-- css/
|   |-- js/
|   '-- views/
|-- routes/
|-- storage/
|-- tests/
|-- vendor/
|-- .env
|-- .env.example
|-- artisan
|-- composer.json
|-- composer.lock
|-- package.json
|-- phpunit.xml
|-- README.md
'-- vite.config.js

```

5.1.2. Explicación de las nuevas carpetas y archivos

Carpeta o archivo	Funcionalidad
app/	Contiene la parte principal de la lógica de Laravel. Aquí suelen estar los modelos, controladores y otros elementos internos de la aplicación. En este proyecto sirve para organizar la lógica relacionada con usuarios, incidencias, estados y gestión técnica.
bootstrap/	Incluye archivos que Laravel usa para arrancar la aplicación. No es la carpeta de Bootstrap CSS, sino la parte interna que prepara el framework antes de ejecutar una petición.
config/	Guarda los archivos de configuración del proyecto, como la configuración de la aplicación, base de datos, sesiones, correo y otros servicios.
database/	Agrupar todo lo relacionado con la base de datos. Sustituye a los scripts SQL sueltos de la versión PHP y permite controlar mejor la creación de tablas y datos iniciales.
database/factories/	Sirve para crear datos de prueba automáticamente. Es útil cuando se quieren generar usuarios o incidencias falsas para comprobar el funcionamiento de la web.
database/migrations/	Contiene las migraciones, que son archivos PHP para crear o modificar tablas. En vez de ejecutar manualmente <code>creacion_tablas.sql</code> , Laravel puede crear la base de datos con comandos como <code>php artisan migrate</code> .

Carpeta o archivo	Funcionalidad
<code>database/seeders/</code>	Contiene los seeders, que insertan datos iniciales en la base de datos. Cumplen una función parecida a <code>sql/datos.sql</code> , por ejemplo para añadir roles, estados, prioridades, barrios o tipos de incidencia.
<code>database/database.sqlite</code>	Archivo de base de datos SQLite. Permite usar una base de datos local sencilla sin depender siempre de MySQL durante el desarrollo.
<code>public/</code>	Es la carpeta pública de la aplicación. Desde aquí se sirve la web al navegador y aquí se colocan recursos accesibles directamente, como imágenes, CSS y JavaScript. En nuestro caso, el CSS propio se ha colocado en <code>public</code> porque así el navegador puede cargarlo directamente y no es necesario compilarlo con Vite.
<code>resources/</code>	Contiene los recursos fuente de la aplicación, sobre todo las vistas, estilos y scripts antes de ser procesados. En este proyecto se utiliza principalmente para las vistas Blade.
<code>resources/css/</code>	Carpeta que Laravel suele usar para estilos que después se compilan con Vite. En nuestro caso no se ha usado como ubicación principal del CSS, ya que se ha preferido dejar los estilos en <code>public</code> para simplificar la carga.
<code>resources/js/</code>	Carpeta para el JavaScript del proyecto. Sirve para añadir interactividad o preparar scripts que después se compilan.
<code>resources/views/</code>	Contiene las vistas Blade. Sustituye a las páginas PHP mezcladas con HTML. Aquí estarían las pantallas como inicio, listado de incidencias, detalle, login, registro, panel técnico y formularios.
<code>routes/</code>	Guarda las rutas de la aplicación. Aquí se define qué ocurre cuando el usuario entra en una URL, por ejemplo la ruta para ver incidencias, reportar una incidencia o acceder al panel técnico.
<code>storage/</code>	Guarda archivos generados por la aplicación, como logs, caché, sesiones o archivos subidos. En el proyecto puede usarse para almacenar imágenes asociadas a incidencias.
<code>tests/</code>	Carpeta para pruebas automáticas. Sirve para comprobar que las funcionalidades importantes siguen funcionando después de hacer cambios.
<code>vendor/</code>	Contiene las dependencias instaladas con Composer, incluido Laravel y sus paquetes. No se modifica normalmente a mano.
<code>.env</code>	Archivo de configuración local. Aquí se indican datos como el nombre de la aplicación, el entorno, la conexión de base de datos y otras claves.
<code>.env.example</code>	Plantilla del archivo <code>.env</code> . Sirve para que otra persona sepa qué variables necesita configurar.

Carpeta o archivo	Funcionalidad
<code>artisan</code>	Herramienta de comandos de Laravel. Se usa para ejecutar migraciones, crear controladores, levantar el servidor y otras tareas del proyecto.
<code>composer.json</code>	Define las dependencias PHP del proyecto y la configuración de Composer.
<code>composer.lock</code>	Guarda las versiones exactas de las dependencias instaladas para que el proyecto sea reproducible.
<code>package.json</code>	Define dependencias de Node.js, normalmente relacionadas con Vite, CSS y JavaScript.
<code>phpunit.xml</code>	Archivo de configuración para ejecutar pruebas con PHPUnit.
<code>vite.config.js</code>	Configuración de Vite, que se encarga de compilar recursos como CSS y JavaScript.

Una decisión importante en esta versión fue colocar el archivo CSS dentro de `public`. De esta forma, el archivo queda disponible directamente para el navegador mediante una ruta pública. Si se hubiese colocado dentro de `resources/css`, lo normal habría sido procesarlo con Vite para generar el archivo final. Como el objetivo era mantener una estructura sencilla y parecida a la versión PHP inicial, se decidió dejar el CSS en `public` para evitar ese paso de compilación.

5.2. Rutas, métodos GET/POST y middleware

En Laravel, las rutas se definen en la carpeta `routes`. En este proyecto se usan para indicar qué controlador debe ejecutarse cuando el usuario entra en una dirección concreta. A diferencia de la versión PHP inicial, el usuario ya no accede directamente a archivos como `login.php` o `incidencias.php`, sino a rutas gestionadas por Laravel.

El archivo de rutas usado en la parte dinámica es el siguiente:

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\HomeController;
use App\Http\Controllers\AuthController;
use App\Http\Controllers\IncidenciaController;

Route::get('/', [HomeController::class, 'index'])->name('home');

Route::get('/contacto', [HomeController::class, 'contacto'])->name('contacto');

Route::get('/incidencias', [IncidenciaController::class, 'index'])
    ->name('incidencias.index');
Route::get('/incidencias/{incidencia}', [IncidenciaController::class, 'show'])
    ->name('incidencias.show');

Route::get('/login', [AuthController::class, 'showLogin'])->name('login');
```

```

Route::post('/login', [AuthController::class, 'login'])->name('login.post');

Route::get('/registro', [AuthController::class, 'showRegister'])->name('register');
Route::post('/registro', [AuthController::class, 'register'])->name('register.post');

Route::post('/logout', [AuthController::class, 'logout'])->name('logout');

Route::middleware('auth')->group(function () {
    Route::get('/reportar', [IncidenciaController::class, 'create'])
        ->name('incidencias.create');
    Route::post('/reportar', [IncidenciaController::class, 'store'])
        ->name('incidencias.store');

    Route::get('/mis-incidencias', [IncidenciaController::class, 'mine'])
        ->name('incidencias.mine');

    Route::get('/panel-tecnico', [IncidenciaController::class, 'panelTecnico'])
        ->name('tecnico.panel');
    Route::post('/panel-tecnico/{incidencia}/estado',
        [IncidenciaController::class, 'cambiarEstado'])->name('tecnico.estado');
});

```

En este archivo aparecen dos métodos HTTP principales: GET y POST. Las rutas con `Route::get()` se usan para mostrar páginas o consultar información. Por ejemplo, `/incidencias` muestra el listado de incidencias, `/login` muestra el formulario de inicio de sesión y `/registro` muestra el formulario de registro.

Las rutas con `Route::post()` se usan cuando el usuario envía datos al servidor. Por ejemplo, `/login` con POST comprueba el correo y la contraseña, `/registro` con POST crea un nuevo usuario, `/reportar` con POST guarda una incidencia y `/panel-tecnico/{incidencia}/estado` con POST actualiza el estado de una incidencia.

También es importante el uso de nombres de ruta mediante `name()`. Por ejemplo, `name('incidencias.index')` permite referirse a esa ruta desde las vistas Blade o desde los controladores sin escribir directamente la URL. Esto facilita cambiar una dirección en el futuro sin tener que modificar todos los enlaces del proyecto.

Otro elemento importante es el parámetro dinámico `{incidencia}`. En rutas como `/incidencias/{incidencia}` o `/panel-tecnico/{incidencia}/estado`, Laravel recibe una incidencia concreta y la pasa al controlador. Esto permite mostrar el detalle de una incidencia o cambiar el estado de una incidencia específica.

Finalmente, se usa `Route::middleware('auth')->group(...)` para proteger un grupo de rutas. El `middleware auth` comprueba que el usuario haya iniciado sesión antes de permitir el acceso. Por eso rutas como `/reportar`, `/mis-incidencias` y `/panel-tecnico` no son públicas. Si el usuario no está autenticado, Laravel no le permite entrar directamente a esas páginas.

Gracias a esta organización, las rutas públicas quedan separadas de las rutas privadas. Las páginas como inicio, contacto, listado de incidencias, detalle, login y registro pueden consultarse sin sesión, mientras que reportar incidencias, ver las incidencias propias y entrar al panel técnico requieren estar identificado.

5.3. Modelo, vista y controlador en Laravel

En la parte dinámica se usa la organización Modelo-Vista-Controlador, también conocida como MVC. Esta forma de organizar el proyecto permite separar los datos, la interfaz y la lógica de la aplicación.

- **Modelo:** representa los datos y las relaciones de la base de datos. Cada modelo suele estar asociado a una tabla. Por ejemplo, `Incidencia`, `TipoIncidencia`, `Barrio`, `EstadoIncidencia` y `User`. Gracias a los modelos, Laravel puede consultar, crear, actualizar o eliminar registros sin escribir consultas SQL directamente en las vistas.
- **Vista:** son los archivos Blade que muestran la interfaz al usuario. En las vistas se coloca el HTML de las pantallas, mezclado con la sintaxis de Blade cuando hace falta mostrar datos dinámicos. Algunos ejemplos son `incidencias/create.blade.php`, `incidencias/show.blade.php`, `tecnico/panel.blade.php` o `contacto.blade.php`.
- **Controlador:** recibe las peticiones del usuario, valida los datos, consulta los modelos y devuelve las vistas correspondientes. Por ejemplo, `IncidenciaController` gestiona crear incidencias, listarlas, mostrar detalles y cambiar estados.

El flujo sería el siguiente: el usuario entra en una ruta, la ruta llama a un controlador, el controlador usa uno o varios modelos para trabajar con la base de datos y finalmente devuelve una vista Blade con la información preparada. Así la aplicación queda más ordenada que en la versión PHP inicial, donde muchas veces la lógica y el HTML estaban juntos en el mismo archivo.

Como ejemplo de controlador, se puede comentar el archivo `AuthController`, encargado de gestionar el inicio de sesión, el registro y el cierre de sesión de los usuarios:

```
<?php

namespace App\Http\Controllers;

use App\Models\Role;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;

class AuthController extends Controller
{
    public function showLogin()
    {
        return view('auth.login');
    }

    public function login(Request $request)
    {
        $datos = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required'],
        ]);
```

```

        if (Auth::attempt($datos)) {
            $request->session()->regenerate();

            if (Auth::user()->esTecnico()) {
                return redirect()->route('tecnico.panel');
            }

            return redirect()->route('home');
        }

        return back()->withErrors([
            'email' => 'El email o la contraseña no son correctos.',
        ])->onlyInput('email');
    }

    public function showRegister()
    {
        return view('auth.register');
    }

    public function register(Request $request)
    {
        $datos = $request->validate([
            'name' => ['required', 'string', 'max:100'],
            'apellidos' => ['required', 'string', 'max:150'],
            'email' => ['required', 'email', 'unique:users,email'],
            'telefono' => ['required', 'string', 'max:20'],
            'password' => ['required', 'string', 'min:4', 'confirmed'],
        ]);

        $rolCiudadano = Role::where('nombre', 'ciudadano')->first();

        $usuario = User::create([
            'role_id' => $rolCiudadano->id,
            'name' => $datos['name'],
            'apellidos' => $datos['apellidos'],
            'email' => $datos['email'],
            'telefono' => $datos['telefono'],
            'password' => Hash::make($datos['password']),
            'activo' => true,
        ]);

        Auth::login($usuario);

        return redirect()->route('home');
    }

    public function logout(Request $request)
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('home');
    }
}

```

En primer lugar, este controlador importa los modelos **Role** y **User**, que se utilizan para consultar el rol ciudadano y crear usuarios nuevos. También importa **Auth**, que gestiona la autenticación, y **Hash**, que permite cifrar la contraseña antes de guardarla en la base de datos.

El método **showLogin()** simplemente devuelve la vista **auth.login**. Es decir, su función es mostrar el formulario de inicio de sesión al usuario. Después, el método **login()** recibe los datos enviados por ese formulario y los valida. El correo debe tener formato de email y la contraseña es obligatoria.

Una vez validados los datos, se usa **Auth::attempt(\$datos)** para intentar iniciar sesión. Si los datos son correctos, se regenera la sesión con **session()->regenerate()**, lo que mejora la seguridad al evitar reutilizar una sesión anterior. Después se comprueba si el usuario tiene rol técnico mediante **esTecnico()**. Si es técnico, se redirige al panel técnico; si no, se redirige a la página principal.

Si el inicio de sesión falla, el controlador devuelve al usuario al formulario anterior usando **back()** y muestra un error indicando que el correo o la contraseña no son correctos. Además, con **onlyInput('email')** se conserva el email escrito para que el usuario no tenga que volver a introducirlo.

El método **showRegister()** devuelve la vista **auth.register**, que contiene el formulario de registro. El método **register()** valida los datos del nuevo usuario: nombre, apellidos, email único, teléfono y contraseña confirmada. Después busca el rol **ciudadano** en la tabla **roles** y crea el usuario con ese rol.

Un punto importante es que la contraseña no se guarda directamente, sino que se cifra con **Hash::make(\$datos['password'])**. Esto es necesario para no almacenar contraseñas en texto plano. Tras crear el usuario, se inicia sesión automáticamente con **Auth::login(\$usuario)** y se redirige al inicio.

Por último, el método **logout()** cierra la sesión con **Auth::logout()**, invalida la sesión actual y regenera el token CSRF. Esto deja la sesión cerrada correctamente y evita que se pueda reutilizar información anterior de la sesión.

5.4. Cómo se han hecho las migraciones

En la parte de Laravel, la base de datos no se crea escribiendo directamente un archivo SQL como en la versión PHP normal, sino mediante migraciones. Las migraciones son archivos PHP que describen las tablas, sus columnas y sus relaciones. Esto permite crear la base de datos de forma automática y ordenada usando el comando **php artisan migrate**.

Para esta parte se creó una migración encargada de generar las tablas relacionadas con usuarios, roles, recuperación de contraseñas y sesiones. La migración usa la clase **Schema** de Laravel y el objeto **Blueprint** para definir cada tabla.

Dentro del método **up()** se indica lo que Laravel debe crear cuando se ejecuta la migración. Como ejemplo, una de las migraciones usadas para crear usuarios, roles, tokens de recuperación y sesiones es la siguiente:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
```

```

use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('roles', function (Blueprint $table) {
            $table->id();
            $table->string('nombre', 50)->unique();
            $table->string('descripcion')->nullable();
            $table->timestamps();
        });

        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->foreignId('role_id')->constrained('roles');
            $table->string('name');
            $table->string('apellidos');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('telefono', 20);
            $table->string('password');
            $table->boolean('activo')->default(true);
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('sessions');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('users');
        Schema::dropIfExists('roles');
    }
};

```

A partir de este ejemplo se pueden destacar varios campos importantes:

- `id()`: crea la clave primaria autoincremental de cada tabla. Laravel la usa para identificar cada registro de forma única.

- **nombre**: en la tabla **roles** guarda el nombre del rol. Se limita a 50 caracteres y se marca como `unique()` para que no se repitan roles con el mismo nombre.
- **descripcion**: permite guardar una explicación del rol. Se define como `nullable()` porque no es obligatorio rellenarla.
- **role_id**: en la tabla **users** es una clave externa que apunta a **roles**. Con `constrained('roles')` se indica que cada usuario debe estar asociado a un rol existente.
- **email**: se marca como `unique()` para evitar que dos usuarios se registren con el mismo correo.
- **email_verified_at**: almacena la fecha de verificación del correo. Puede ser nulo porque un usuario puede existir aunque todavía no haya verificado su email.
- **password**: guarda la contraseña del usuario, normalmente cifrada mediante los mecanismos de Laravel.
- **activo**: indica si el usuario está habilitado. Se define con `default(true)`, por lo que un usuario nuevo queda activo por defecto.
- **rememberToken()**: crea el campo usado por Laravel para mantener sesiones recordadas cuando el usuario marca la opción de recordar sesión.
- **timestamps()**: crea automáticamente los campos **created_at** y **updated_at**, que sirven para saber cuándo se creó y cuándo se modificó cada registro.
- **password_reset_tokens**: guarda el email, el token y la fecha de creación para poder recuperar contraseñas.
- **sessions**: almacena sesiones en base de datos, incluyendo el usuario asociado, la IP, el navegador, los datos de sesión y la última actividad.

Después de ver el ejemplo, dentro del método `up()` se puede resumir lo que Laravel crea cuando se ejecuta la migración:

- Primero se crea la tabla **roles**, que guarda los tipos de usuario de la aplicación, como ciudadano o técnico. Tiene un campo **id**, un campo **nombre** único, una **descripcion** opcional y los campos automáticos de fecha con `timestamps()`.
- Después se crea la tabla **users**, que guarda los usuarios registrados. Esta tabla tiene una clave externa **role_id** que se relaciona con la tabla **roles**. Así, cada usuario pertenece a un rol concreto.
- En **users** también se guardan datos como **name**, **apellidos**, **email**, **telefono**, **password** y el campo **activo**, que indica si el usuario está habilitado.
- Luego se crea **password_reset_tokens**, que Laravel usa para gestionar tokens de recuperación de contraseña asociados a un email.
- Finalmente se crea la tabla **sessions**, que permite guardar sesiones en la base de datos. Esta tabla almacena el identificador de sesión, el usuario asociado si existe, la IP, el navegador, los datos de sesión y la última actividad.

La relación más importante de esta migración está en la tabla `users`, concretamente en la línea donde se define `foreignId('role_id')->constrained('roles')`. Con esto Laravel crea una clave externa que conecta cada usuario con un registro de la tabla `roles`. Gracias a esta relación se puede saber si un usuario es ciudadano o técnico y mostrarle opciones diferentes dentro de la aplicación.

También se define el método `down()`, que hace lo contrario que `up()`. Si se quiere deshacer la migración, Laravel elimina las tablas en orden inverso: primero `sessions`, después `password_reset_tokens`, luego `users` y por último `roles`. Este orden es importante porque `users` depende de `roles` mediante una clave externa.

En resumen, las migraciones se han usado para pasar de una base de datos creada manualmente con SQL a una base de datos definida desde Laravel. Esto hace que el proyecto sea más fácil de montar en otro ordenador, porque basta con configurar el archivo `.env` y ejecutar las migraciones para crear las tablas necesarias.

Además, también se han aprovechado las migraciones para mejorar la base de datos existente añadiendo comprobaciones antes de crear tablas o columnas. Esto es útil porque evita errores si una tabla ya existe o si una columna ya se había añadido previamente. Como ejemplo, se creó una migración para incorporar la tabla `barrios` y ampliar la tabla `incidencias` con nuevos campos relacionados con la ubicación:

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        if (!Schema::hasTable('barrios')) {
            Schema::create('barrios', function (Blueprint $table) {
                $table->id();
                $table->string('nombre', 100);
                $table->string('distrito', 100)->nullable();
                $table->string('codigo_postal', 10)->nullable();
                $table->timestamps();
            });
        }

        Schema::table('incidencias', function (Blueprint $table) {
            if (!Schema::hasColumn('incidencias', 'barrio_id')) {
                $table->foreignId('barrio_id')->nullable()
                    ->after('estado_incidencia_id')->constrained('barrios');
            }

            if (!Schema::hasColumn('incidencias', 'codigo_postal')) {
                $table->string('codigo_postal', 10)->nullable()->after('direccion');
            }

            if (!Schema::hasColumn('incidencias', 'fecha_incidencia')) {
                $table->date('fecha_incidencia')->nullable()->after('codigo_postal');
            }
        });
    }
};
```

```

    }

    public function down(): void
    {
        Schema::table('incidencias', function (Blueprint $table) {
            if (Schema::hasColumn('incidencias', 'barrio_id')) {
                $table->dropForeign(['barrio_id']);
                $table->dropColumn('barrio_id');
            }

            if (Schema::hasColumn('incidencias', 'codigo_postal')) {
                $table->dropColumn('codigo_postal');
            }

            if (Schema::hasColumn('incidencias', 'fecha_incidencia')) {
                $table->dropColumn('fecha_incidencia');
            }
        });

        Schema::dropIfExists('barrios');
    }
};

```

En esta migración se utilizan comprobaciones como `Schema::hasTable()` y `Schema::hasColumn()`. La primera comprueba si la tabla `barrios` ya existe antes de crearla. Las segundas comprueban si las columnas `barrio_id`, `codigo_postal` y `fecha_incidencia` ya existen dentro de `incidencias`. De esta forma, la migración es más segura y se evitan errores al ejecutarla sobre una base de datos que ya tiene parte de la estructura creada.

La tabla `barrios` permite guardar información de las zonas de Granada, incluyendo el `nombre`, el `distrito` y el `codigo_postal`. Por otro lado, la tabla `incidencias` se mejora añadiendo `barrio_id`, que relaciona cada incidencia con un barrio; `codigo_postal`, que permite guardar una ubicación más concreta; y `fecha_incidencia`, que indica cuándo ocurrió el problema reportado.

El método `down()` también incluye comprobaciones antes de eliminar columnas. Primero se elimina la clave externa de `barrio_id` y después la columna, ya que no se puede borrar directamente una columna que tiene una restricción activa. Finalmente, se elimina la tabla `barrios` con `dropIfExists()`, lo que permite deshacer la migración de forma ordenada.

En resumen, además de crear la estructura inicial, las migraciones permiten mejorar la base de datos de forma controlada, añadiendo nuevas tablas, nuevas columnas y comprobaciones para evitar conflictos.

5.5. Funcionamiento de la parte dinámica

En Laravel el funcionamiento queda más ordenado. El usuario ya no entra directamente a archivos PHP concretos, sino a rutas definidas por la aplicación. Por ejemplo, una ruta puede mostrar todas las incidencias, otra puede abrir el formulario de registro y otra puede permitir que el técnico cambie el estado de una incidencia.

Cuando se recibe una petición, Laravel sigue este flujo:

1. El navegador solicita una URL del proyecto.
2. Laravel busca esa URL en los archivos de `routes`.
3. La ruta llama al controlador correspondiente dentro de `app`.
4. El controlador usa modelos para consultar o modificar la base de datos.
5. Si hace falta mostrar una página, se carga una vista Blade desde `resources/views`.
6. La vista genera el HTML final que ve el usuario.

Gracias a esto, la web deja de ser un conjunto de páginas PHP separadas y pasa a ser una aplicación dinámica organizada por capas. Esto facilita añadir nuevas funcionalidades, modificar la base de datos con migraciones, reutilizar plantillas Blade y mantener separada la lógica del diseño.

5.6. Diseño, accesibilidad y responsive

En el diseño se ha intentado mejorar la accesibilidad usando una estructura clara, colores con buen contraste y elementos visuales fáciles de distinguir. Los textos principales usan colores oscuros sobre fondos claros, y los botones y enlaces tienen estilos diferenciados para que el usuario pueda reconocerlos fácilmente.

También se ha usado un diseño responsive mediante `@media (max-width: 768px)`, pensado para móviles y tablets. Con esta regla CSS, la web puede adaptar sus elementos cuando la pantalla es más pequeña, reorganizando contenidos y evitando que formularios, tarjetas o tablas se vean mal en dispositivos móviles.

Otra decisión importante ha sido separar el HTML del CSS. Los estilos no están mezclados directamente dentro de las vistas, sino colocados en un archivo CSS aparte. Esto permite mantener las vistas Blade más limpias y facilita modificar colores, márgenes, botones o formularios desde un único lugar.

En cuanto al contraste, se ha usado una combinación razonable de texto oscuro sobre fondos claros. Esto ayuda a que los contenidos principales sean fáciles de leer y a que los apartados importantes destaquen visualmente.

Los formularios incluyen etiquetas visibles mediante `label` y clases como `.form-label`. Además, los campos tienen estados de *focus* visibles, por lo que el usuario puede identificar claramente en qué campo está escribiendo. Esto mejora la experiencia de uso, especialmente al iniciar sesión, registrarse o crear una incidencia.

También se ha añadido el atributo `alt` en las imágenes. Este texto alternativo permite describir el contenido de una imagen cuando no se puede cargar correctamente y mejora la accesibilidad para usuarios que utilizan lectores de pantalla.

Los enlaces y botones tienen efectos visuales de *hover*. Esto sirve para indicar interacción cuando el usuario pasa el ratón por encima y ayuda a diferenciar los elementos pulsables del resto del contenido.

La estructura general de la web se ha organizado de forma clara, separando *navbar*, contenido principal y *footer*. Además, las tarjetas, tablas y avisos se presentan con márgenes, sombras y bordes redondeados para facilitar la lectura.

En el panel técnico, la información se muestra en tablas estructuradas. Las tablas tienen encabezados con `<th>`, lo que es importante para organizar la información por columnas y facilitar una lectura más ordenada. También se usan mensajes visibles, como `alert`, *badges* y estados, para indicar errores, avisos o el estado actual de una incidencia.

5.7. Funciones disponibles en la parte dinámica

En la versión dinámica de Laravel también se puede acceder con un usuario técnico para comprobar la parte de administración de incidencias. Este usuario se crea como dato inicial mediante los seeders de Laravel, por lo que tanto el correo como la contraseña quedan definidos en el seeder correspondiente. Los datos de acceso son:

- **Correo:** `tecnico@granada.es`
- **Contraseña:** 1234, definida en el seeder del usuario técnico.

Al iniciar sesión con este usuario, se puede entrar al Panel Técnico. Desde este panel el técnico puede ver todas las incidencias registradas en la aplicación y revisar la información principal de cada una.

Los datos que se muestran de cada incidencia son:

- Título de la incidencia.
- Dirección donde ocurre.
- Barrio asociado.
- Tipo de incidencia.
- Usuario que la reportó.
- Estado actual.
- Fotografía

Además, el técnico puede entrar al detalle de una incidencia concreta. Desde esa pantalla puede consultar mejor la información del aviso y cambiar el estado de la incidencia usando un desplegable. Después de seleccionar el nuevo estado, se guarda el cambio para que la incidencia quede actualizada en la base de datos.

La parte dinámica también permite crear nuevas incidencias desde el formulario correspondiente. En ese formulario el usuario puede escribir el título, la descripción, la dirección y el resto de datos necesarios, asignar el tipo de incidencia, seleccionar el barrio y añadir fotografías. Al enviar el formulario, Laravel valida los datos, guarda la incidencia y almacena las imágenes asociadas para que puedan verse después en el detalle.

6. Conclusiones

Con este proyecto se ha conseguido desarrollar una aplicación completa para la gestión de incidencias urbanas. La primera versión, realizada con PHP, HTML, Bootstrap, CSS y MySQL, permitió construir la funcionalidad básica: mostrar incidencias, registrar usuarios, iniciar sesión, crear avisos y gestionar cambios de estado desde el panel técnico.

La base de datos ha sido una parte fundamental del trabajo, ya que organiza la información mediante tablas relacionadas. La tabla **incidencias** actúa como elemento central y se conecta con usuarios, roles, tipos de incidencia, estados, prioridades, barrios, imágenes, historial y notificaciones. Gracias a esta estructura, la información queda mejor organizada y se evitan repeticiones innecesarias.

La segunda versión con Laravel ha permitido mejorar la organización del proyecto. Al usar rutas, controladores, modelos, migraciones y vistas Blade, la aplicación queda dividida en partes más claras. Esto facilita el mantenimiento, permite ampliar funcionalidades de forma más ordenada y separa mejor la lógica de la interfaz.

También se han aprovechado las migraciones para crear y mejorar la base de datos desde el propio framework. Esto hace que el proyecto sea más fácil de instalar en otro entorno, ya que las tablas pueden generarse automáticamente con los comandos de Laravel. Además, las comprobaciones con `Schema::hasTable()` y `Schema::hasColumn()` permiten hacer cambios de forma más segura.

En cuanto al diseño, se ha intentado que la web sea clara, responsive y accesible. Se han usado colores con contraste, etiquetas en formularios, estados *focus*, efectos *hover*, textos alternativos en imágenes y una estructura visual ordenada con menús, tablas, tarjetas y avisos.

En conclusión, el paso de la versión estática a la versión dinámica ha servido para transformar una web funcional en una aplicación más organizada y mantenible. Laravel ha ayudado a estructurar mejor el código y a preparar el proyecto para futuras mejoras, como añadir más roles, ampliar el panel técnico o incorporar nuevas funcionalidades para los ciudadanos.